Tutorial NGINX + Python + SFTP

Interface Web para acesso a um servidor SFTP



Campus Leopoldina

Disciplina: Computação em Nuvem
Professor: Daniel Henriques
Curso: Engenharia de Computação
Data: 31 de agosto de 2026

1 Interface Web para acesso a um servidor SFTP

Nesta prática será criada uma interface Web simples para acesso a um servidor SFTP. O usuário informará, por meio da própria página Web, o endereço IP do servidor SFTP, o nome de usuário, a senha e o diretório remoto que deseja acessar.

A arquitetura utilizada será a seguinte:

```
Navegador
   |
   | HTTP/1.1
   v
NGINX
   |
   | HTTP
   v
Flask
   |
   | SFTP
   v
Servidor SFTP
```

Dessa forma, o NGINX será responsável por receber as conexões HTTP na porta 80 e encaminhá-las para a aplicação Flask, enquanto o Flask realizará a comunicação com o servidor de arquivos utilizando o protocolo SFTP.

1.1 Instalação dos pacotes necessários

No contêiner destinado ao servidor Web, inicialmente devem ser instalados o NGINX, o Flask e a biblioteca Paramiko, utilizada para a comunicação SFTP:

```
apt update
apt install nginx python3-flask python3-paramiko
```

Em seguida, deve-se criar a estrutura de diretórios utilizada pela aplicação:

```
mkdir -p /opt/websftp/templates
cd /opt/websftp
```

A aplicação será composta por um arquivo Python principal e três arquivos HTML:

```
/opt/websftp/
|-- app.py
`-- templates/
    |-- login.html
    |-- arquivos.html
    `-- erro.html
```

1.2 Criação da aplicação Python

Crie o arquivo principal da aplicação:

```
nano /opt/websftp/app.py
```

Insira o seguinte código:

```
from flask import Flask, render_template, request, redirect, session, send_file
import paramiko
import tempfile
import os
from werkzeug.utils import secure_filename

app = Flask(__name__)

# Necessário para o Flask utilizar sessões.
# Para esta prática pode ser utilizada uma chave simples.
app.secret_key = "aula-computacao-em-nuvem"


def conectar_sftp():

    host = session["host"]
    usuario = session["usuario"]
    senha = session["senha"]

    transporte = paramiko.Transport((host, 22))

    transporte.connect(
        username=usuario,
        password=senha
    )

    sftp = paramiko.SFTPClient.from_transport(transporte)
```

```
    return transporte, sftp


@app.route("/", methods=["GET", "POST"])
def login():

    if request.method == "POST":

        session["host"] = request.form["host"]
        session["usuario"] = request.form["usuario"]
        session["senha"] = request.form["senha"]
        session["diretorio"] = request.form["diretorio"]

        try:

            transporte, sftp = conectar_sftp()

            # Testa se o diretório informado existe
            sftp.listdir(session["diretorio"])

            sftp.close()
            transporte.close()

            return redirect("/arquivos")

        except Exception as erro:

            return render_template(
                "login.html",
                erro=str(erro)
            )

    return render_template("login.html")


@app.route("/arquivos")
def arquivos():

    if "host" not in session:
        return redirect("/")

    try:

        transporte, sftp = conectar_sftp()

        lista = sftp.listdir(session["diretorio"])
```

```
        sftp.close()
        transporte.close()

        return render_template(
            "arquivos.html",
            arquivos=lista,
            host=session["host"],
            diretorio=session["diretorio"]
        )

    except Exception as erro:

        return render_template(
            "erro.html",
            erro=str(erro)
        )


@app.route("/upload", methods=["POST"])
def upload():

    if "host" not in session:
        return redirect("/")

    arquivo = request.files["arquivo"]

    if arquivo.filename == "":
        return redirect("/arquivos")

    nome = secure_filename(arquivo.filename)

    transporte, sftp = conectar_sftp()

    caminho_remoto = (
        session["diretorio"].rstrip("/")
        + "/"
        + nome
    )

    with tempfile.NamedTemporaryFile(delete=False) as temp:

        arquivo.save(temp.name)

        caminho_temporario = temp.name

    sftp.put(
```

```
        caminho_temporario,
        caminho_remoto
    )

    os.remove(caminho_temporario)

    sftp.close()
    transporte.close()

    return redirect("/arquivos")


@app.route("/download/<nome>")
def download(nome):

    if "host" not in session:
        return redirect("/")

    nome = secure_filename(nome)

    transporte, sftp = conectar_sftp()

    caminho_remoto = (
        session["diretorio"].rstrip("/")
        + "/"
        + nome
    )

    temp = tempfile.NamedTemporaryFile(delete=False)

    caminho_temporario = temp.name

    temp.close()

    sftp.get(
        caminho_remoto,
        caminho_temporario
    )

    sftp.close()
    transporte.close()

    return send_file(
        caminho_temporario,
        as_attachment=True,
        download_name=nome
    )
```

```
@app.route("/sair")
def sair():

    session.clear()

    return redirect("/")


if __name__ == "__main__":

    app.run(
        host="127.0.0.1",
        port=5000
    )
```

A aplicação utiliza a biblioteca Paramiko para estabelecer uma conexão SFTP. Os dados de conexão não ficam definidos diretamente no código. Em vez disso, são informados pelo usuário na página Web e armazenados temporariamente na sessão do Flask.

As principais operações SFTP utilizadas são:

- `sftp.listdir()` para listar os arquivos;
- `sftp.put()` para enviar arquivos;
- `sftp.get()` para baixar arquivos.

1.3 Criação da página de conexão

Crie o arquivo:

```
nano /opt/websftp/templates/login.html
```

Insira o seguinte conteúdo:

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="UTF-8">

    <title>Conectar ao servidor SFTP</title>

</head>
```

```
<body>

    <h1>Servidor de Arquivos</h1>

    <hr>

    <h2>Conectar ao servidor SFTP</h2>

    {% if erro %}

        <p>
            <b>Erro:</b> {{ erro }}
        </p>

    {% endif %}

    <form method="post">

        <p>
            Servidor SFTP:
            <br>

            <input
                type="text"
                name="host"
                placeholder="192.168.1.115"
                required>
        </p>

        <p>
            Usuário:
            <br>

            <input
                type="text"
                name="usuario"
                required>
        </p>

        <p>
            Senha:
            <br>

            <input
                type="password"
                name="senha"
                required>
```

```
        </p>

        <p>
            Diretório remoto:
            <br>

            <input
                type="text"
                name="diretorio"
                value="/home/arquivos/dados"
                required>
        </p>

        <input
            type="submit"
            value="Conectar">

    </form>

</body>

</html>
```

Essa página será exibida inicialmente e permitirá ao usuário informar os dados necessários para a conexão SFTP.

A interface terá, aproximadamente, a seguinte organização:

```
Servidor de Arquivos
-----------------------------------

Conectar ao servidor SFTP

Servidor SFTP:
[192.168.1.115          ]

Usuário:
[arquivos               ]

Senha:
[********               ]

Diretório remoto:
[/home/arquivos/dados   ]

[ Conectar ]
```

1.4 Criação da página de arquivos

Crie o arquivo:

```
nano /opt/websftp/templates/arquivos.html
```

Insira o seguinte conteúdo:

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="UTF-8">

    <title>Arquivos SFTP</title>

</head>

<body>

    <h1>Servidor de Arquivos</h1>

    <hr>

    <p>
        Servidor:
        <b>{{ host }}</b>
    </p>

    <p>
        Diretório:
        <b>{{ diretorio }}</b>
    </p>

    <p>
        <a href="/sair">
            Desconectar
        </a>
    </p>

    <hr>

    <h2>Arquivos disponíveis</h2>

    <ul>
```

```
        {% for arquivo in arquivos %}

            <li>

                {{ arquivo }}

                -

                <a href="/download/{{ arquivo }}">
                    Baixar
                </a>

            </li>

        {% endfor %}

    </ul>

    <hr>

    <h2>Enviar arquivo</h2>

    <form
        action="/upload"
        method="post"
        enctype="multipart/form-data">

        <input
            type="file"
            name="arquivo"
            required>

        <input
            type="submit"
            value="Enviar">

    </form>

</body>

</html>
```

Essa página será exibida após uma conexão SFTP bem-sucedida. Nela, os arquivos existentes no diretório remoto serão apresentados em uma lista.

Para cada arquivo será disponibilizado um link para download. Também será apresentado

um formulário que permite selecionar um arquivo no computador local e enviá-lo para o servidor SFTP.

1.5 Criação da página de erro

Crie o arquivo:

```
nano /opt/websftp/templates/erro.html
```

Insira o seguinte conteúdo:

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="UTF-8">

    <title>Erro</title>

</head>

<body>

    <h1>Erro de conexão</h1>

    <hr>

    <p>
        Não foi possível acessar o servidor SFTP.
    </p>

    <p>
        {{ erro }}
    </p>

    <p>
        <a href="/">
            Voltar
        </a>
    </p>

</body>

</html>
```

Essa página será utilizada para apresentar erros de comunicação com o servidor SFTP.

1.6 Teste da aplicação Flask

Antes de configurar o NGINX, recomenda-se testar a aplicação diretamente.

Execute:

```
cd /opt/websftp
python3 app.py
```

A aplicação deverá indicar que está sendo executada em:

`http://127.0.0.1:5000`

É possível testar a aplicação dentro do próprio contêiner utilizando:

```
curl http://127.0.0.1:5000
```

Como a aplicação está configurada para escutar apenas no endereço `127.0.0.1`, ela não poderá ser acessada diretamente por outros computadores da rede.

O acesso externo será realizado por meio do NGINX.

1.7 Configuração do NGINX

Primeiro, remova a configuração padrão do NGINX:

```
rm /etc/nginx/sites-enabled/default
```

Em seguida, crie um novo arquivo de configuração:

```
nano /etc/nginx/sites-available/websftp
```

Insira:

```
server {

    listen 80;
    listen [::]:80;

    server_name _;

    location / {

        proxy_pass http://127.0.0.1:5000;

        proxy_set_header Host $host;

        proxy_set_header X-Real-IP $remote_addr;

```

```
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for
            ;
    }
}
```

Essa configuração determina que o NGINX escutará conexões HTTP na porta 80 e encaminhará as requisições para a aplicação Flask, executada localmente na porta 5000.

A arquitetura passa a ser:

```
Cliente
   |
   | HTTP/1.1
   | porta 80
   v
NGINX
   |
   | HTTP
   | 127.0.0.1:5000
   v
Flask
```

Para habilitar a nova configuração, crie um link simbólico:

```
ln -s /etc/nginx/sites-available/websftp \
/etc/nginx/sites-enabled/websftp
```

Teste a configuração:

```
nginx -t
```

Caso não sejam apresentados erros, reinicie o servidor:

```
systemctl restart nginx
```

1.8 Configuração da aplicação como serviço

Para evitar que a aplicação Flask precise ser iniciada manualmente a cada reinicialização do contêiner, pode-se criar um serviço `systemd`.

Crie:

```
nano /etc/systemd/system/websftp.service
```

Insira:

```
[Unit]
Description=Interface Web para servidor SFTP
After=network.target

```

```
[Service]
WorkingDirectory=/opt/websftp
ExecStart=/usr/bin/python3 /opt/websftp/app.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Em seguida, execute:

```
systemctl daemon-reload
systemctl enable --now websftp
```

Para verificar o funcionamento do serviço:

```
systemctl status websftp
```

1.9 Acesso à aplicação pela rede

Para descobrir o endereço IP do contêiner Web, utilize:

```
ip addr
```

Um resultado possível seria:

```
eth0    UP    192.168.1.122/24
```

Nesse caso, a aplicação poderá ser acessada a partir de outro computador da rede utilizando:

```
http://192.168.1.122
```

Não é necessário informar explicitamente a porta, pois o protocolo HTTP utiliza a porta 80 por padrão e o NGINX está configurado para escutá-la.

1.10 Fluxo de comunicação

Quando o usuário acessa a aplicação, inicialmente informa:

- endereço IP ou hostname do servidor SFTP;
- nome de usuário;
- senha;
- diretório remoto.

Após a conexão, o fluxo de comunicação ocorre da seguinte forma:

```
Navegador
    |
    | HTTP
    v
```

```
NGINX
    |
    | HTTP
    v
Flask
    |
    | SFTP/SSH
    v
Servidor de arquivos
```

É importante observar que são utilizados dois protocolos de aplicação diferentes:

- HTTP, entre o navegador e o servidor Web;
- SFTP, entre a aplicação Flask e o servidor de arquivos.

1.11 Observação sobre HTTP e segurança

Nesta prática será utilizado HTTP simples, sem HTTPS. Isso significa que os dados transmitidos entre o navegador e o servidor Web não possuem criptografia fornecida pelo protocolo HTTP.

Dessa forma, a arquitetura pode ser representada por:

```
Navegador
    |
    | HTTP
    | sem criptografia
    v
Servidor Web
    |
    | SFTP/SSH
    | criptografado
    v
Servidor de arquivos
```

A comunicação entre a aplicação Flask e o servidor SFTP é protegida pelo SSH, enquanto a comunicação entre o navegador e o servidor Web permanece sem criptografia.

Essa diferença permite observar, de forma prática, a função de protocolos como HTTP, HTTPS, SSH e SFTP em uma aplicação distribuída.

1.12 Observação sobre as credenciais

Nesta implementação, as informações de conexão com o servidor SFTP são armazenadas temporariamente em uma sessão do Flask. Isso evita que o endereço IP, o usuário e a senha precisem ficar diretamente definidos no código Python.

Entretanto, a implementação foi desenvolvida com finalidade didática. O mecanismo padrão

de sessão do Flask utiliza cookies assinados, não sendo recomendado armazenar credenciais sensíveis dessa forma em aplicações reais.

Em ambientes de produção, deve-se utilizar mecanismos apropriados para gerenciamento de credenciais, armazenamento seguro e comunicação utilizando HTTPS.